Traffic-Light Controller (Moore FSM) and MOD-6 Synchronous Counter

Digital Electronics – Basic Electronics course (TieEdu) · Design document

What this document contains. (1) A two-road traffic-light controller designed as a **Moore finite-state machine**: specification, state diagram, state table, state assignment, next-state and output tables, K-map simplification for every flip-flop input and every output, and the final flip-flop/gate circuit. (2) A **MOD-6 synchronous counter** (N = 6, inside the allowed range 5 to 10) with its terminal-count logic and a wrap-around proof. (3) How the two circuits work together (the counter is the FSM's timer), timing diagrams, and a short reflection on why Moore was chosen over Mealy.

Notation. X' means NOT X (drawn as an overbar in the circuit figures); · is AND; + is OR; ⊕ is XOR. All flip-flops are positive-edge-triggered D flip-flops on one common clock, so the D input of each flip-flop equals its next state. A 1 in an output column means the lamp is ON.

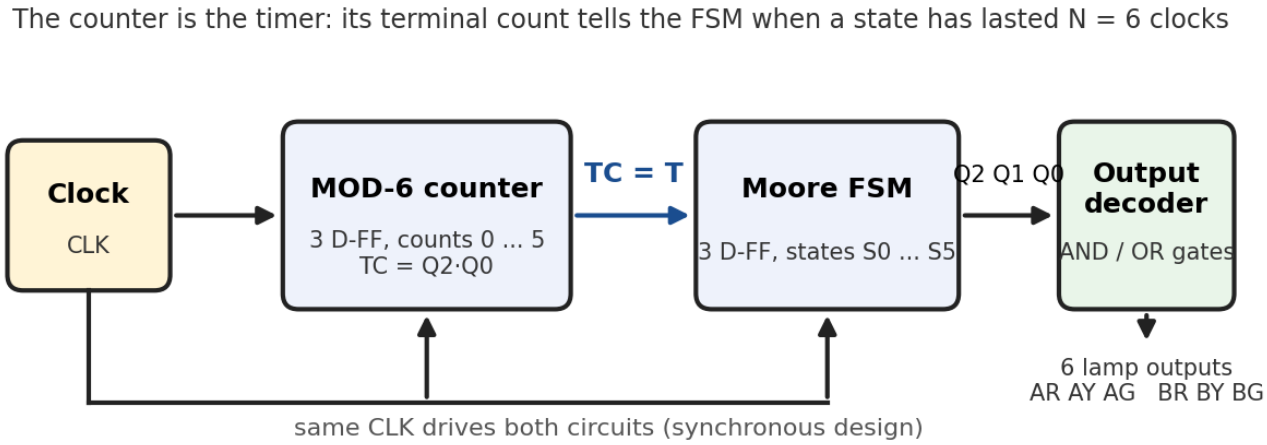


Figure 1 – System overview: MOD-6 counter (timer) → Moore FSM → output decoder.

Part 1 – Traffic-light controller (Moore FSM)

1.1 Specification

• Two roads meet at a junction: **Road A** (main) and **Road B** (side). Each has Red, Yellow and Green lamps, giving six outputs A_R, A_Y, A_G, B_R, B_Y, B_G.

• One input, **T** (“time is up”). While T = 0 the FSM stays in its state; at the clock edge where T = 1 it moves to the next state. T is the terminal count of the MOD-6 counter (Part 2), so every state lasts N = 6 clock periods.

• Green gets two time slots and yellow one, so: Road A green 12 clocks, A yellow 6, Road B green 12, B yellow 6; one full cycle = 36 clocks. With a 2 Hz clock this is 6 s / 3 s / 6 s / 3 s, a cycle of 18 s (the clock frequency is a free design choice).

• **Safety rule:** exactly one lamp lit per road, and when one road has green or yellow the other road must be red. A power-on RESET forces the machine into S0. No separate all-red interval is used; the yellow phase is the clearance time.

1.2 State diagram and state table

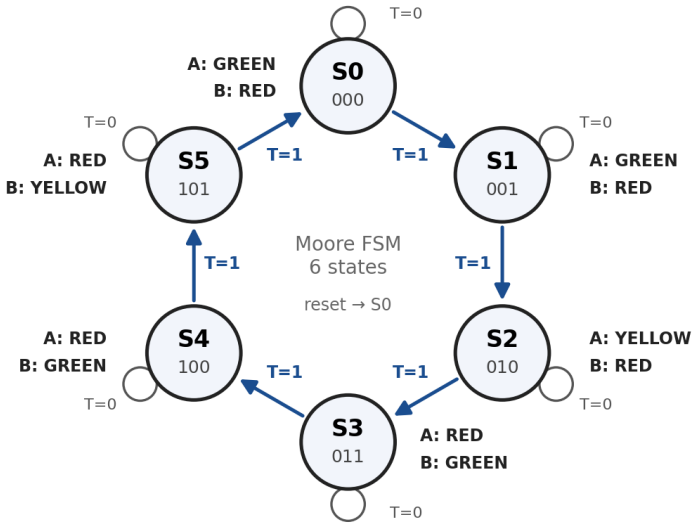


Figure 2 – Moore state diagram. Lamp outputs are attached to the states (inside/next to the circles), not to the arrows.

State	$Q_2Q_1Q_0$	Road A	Road B	Time slots
S0	000	GREEN	RED	1 (6 clocks)
S1	001	GREEN	RED	1
S2	010	YELLOW	RED	1
S3	011	RED	GREEN	1
S4	100	RED	GREEN	1
S5	101	RED	YELLOW	1

Table 1 – State table (Moore: lamp pattern depends on the state only).

1.3 State assignment

Six states need $\lceil \log_2 6 \rceil = 3$ flip-flops ($Q_2Q_1Q_0$). The states are numbered in **binary counting order** (S0 = 000 … S5 = 101) because the machine simply steps through them in a ring. This makes D_0 a plain toggle ($Q_0 \oplus T$), keeps the same logic as a MOD-6 counter gated by T, and leaves two unused codes, 110 and 111, which are used as **don't-cares** to simplify the K-maps. Green is given two states (S0, S1 and S3, S4) so that it lasts twice as long as yellow.

1.4 Present-state / next-state / output table

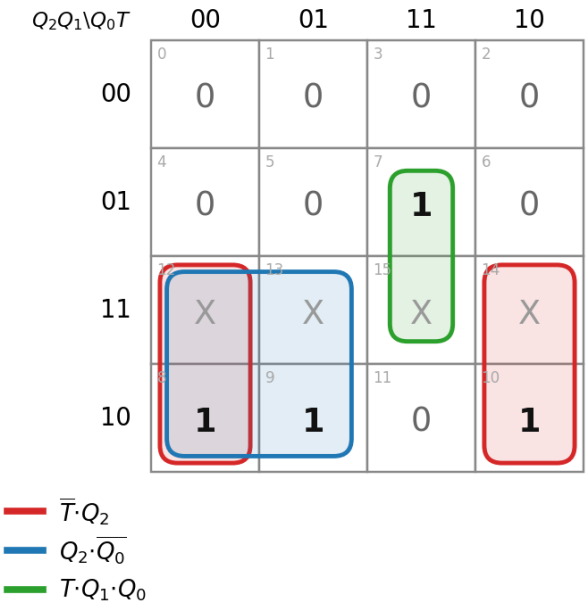
Present state $Q_2Q_1Q_0$	Next, T = 0 $D_2D_1D_0$	Next, T = 1 $D_2D_1D_0$	A_R	A_Y	A_G	B_R	B_Y	B_G
000 (S0)	000	001	0	0	1	1	0	0
001 (S1)	001	010	0	0	1	1	0	0
010 (S2)	010	011	0	1	0	1	0	0
011 (S3)	011	100	1	0	0	0	0	1
100 (S4)	100	101	1	0	0	0	0	1
101 (S5)	101	000	1	0	0	0	1	0
110 (unused)	X X X	X X X	X	X	X	X	X	X
111 (unused)	X X X	X X X	X	X	X	X	X	X

Table 2 – Next-state and output table. For D flip-flops D = next state. X = don't care (states 110 and 111 never occur in normal operation).

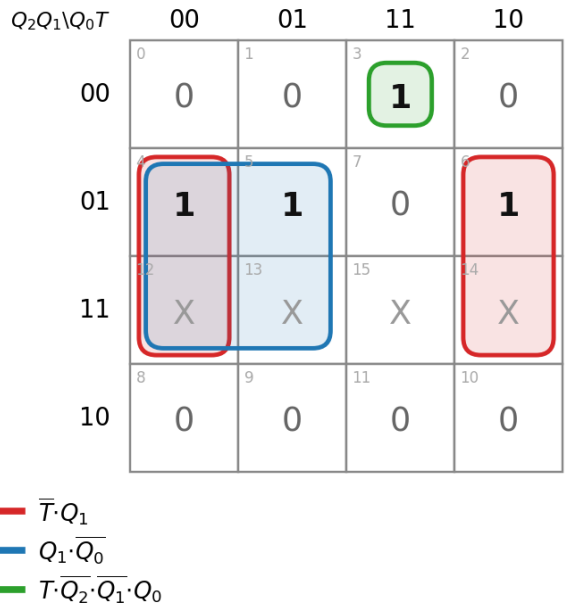
1.5 K-maps for the flip-flop inputs (next-state logic)

Rows are Q_2Q_1, columns are Q_0T, both in Gray-code order. The small grey number in each cell is the minterm number. Coloured loops are the prime-implicant groups used; groups that touch the left and right edge wrap around. The row 11 (states 110/111) is all don't-care.

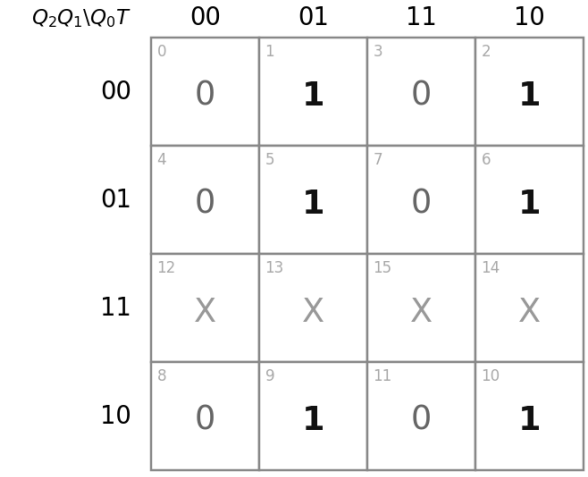
D_2 **(next Q_2)**



D_1 **(next Q_1)**



D_0 **(next Q_0)**



checkerboard of 1s: $D_0 = Q_0 \oplus T$ (no grouping)

Reading the maps.

D_2: hold Q_2 when T = 0 (Q_2T'), keep it when Q_0 = 0 in S4 (Q_2Q_0'), and set it on the S3→S4 step ($T\cdot Q_1\cdot Q_0$).

D_1: hold Q_1 (Q_1T'), keep it through S2 (Q_1Q_0'), and set it on the S1→S2 step ($T\cdot Q_2'\cdot Q_1'\cdot Q_0$).

D_0: toggles on every T = 1.

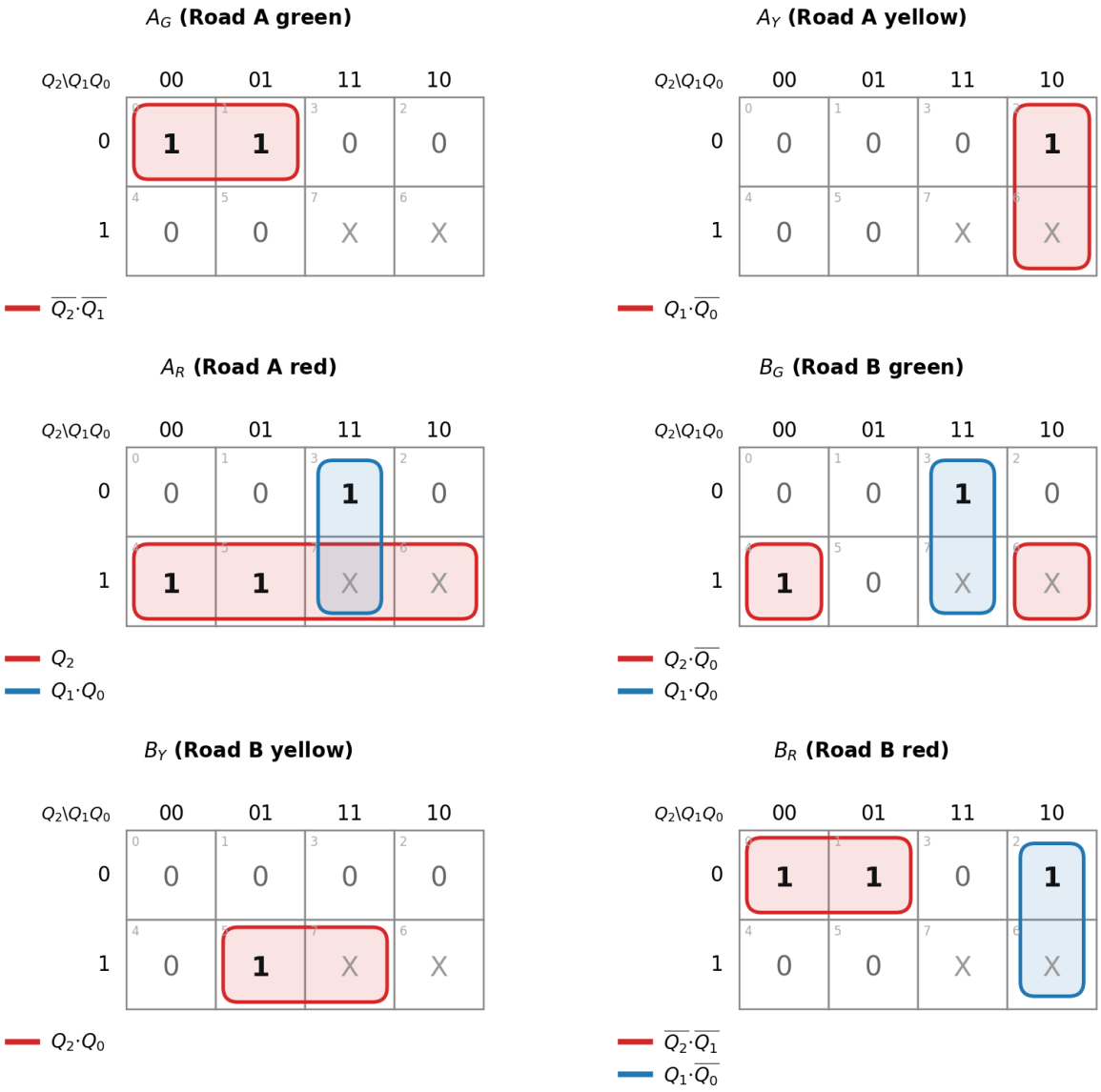
$$D_2 = Q_2\cdot T' + Q_2\cdot Q_0' + T\cdot Q_1\cdot Q_0$$

$$D_1 = Q_1\cdot T' + Q_1\cdot Q_0' + T\cdot Q_2'\cdot Q_1'\cdot Q_0$$

$$D_0 = Q_0 \oplus T$$

1.6 K-maps for the outputs (Moore: functions of $Q_2Q_1Q_0$ only)

Rows are Q_2 (0, 1); columns are Q_1Q_0 in Gray order. Cells 110 and 111 are don't-cares.



Output	Simplified equation	Check
A_G	$Q_2'\cdot Q_1'$	1 in S0, S1
A_Y	$Q_1\cdot Q_0'$	1 in S2 (010); uses X at 110
A_R	$Q_2 + Q_1\cdot Q_0$	1 in S3, S4, S5
B_G	$Q_2\cdot Q_0' + Q_1\cdot Q_0$	1 in S3 (011), S4 (100)
B_Y	$Q_2\cdot Q_0$	1 in S5 (101); uses X at 111
B_R	$Q_2'\cdot Q_1' + Q_1\cdot Q_0'$	1 in S0, S1, S2 (= $A_G + A_Y$)

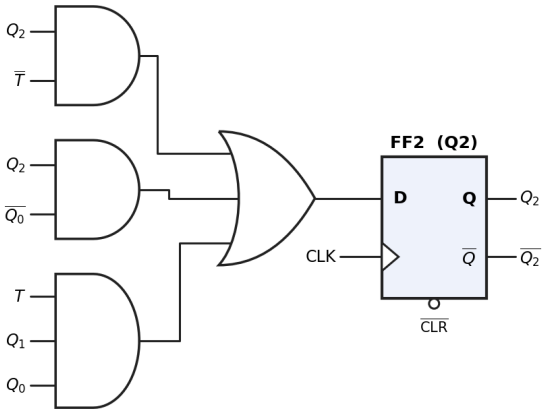
Table 3 – Output equations. Note $A_R = B_G + B_Y$ and $B_R = A_G + A_Y$: a road is red exactly when the other road has the right of way.

1.7 Final circuit: flip-flops and gates

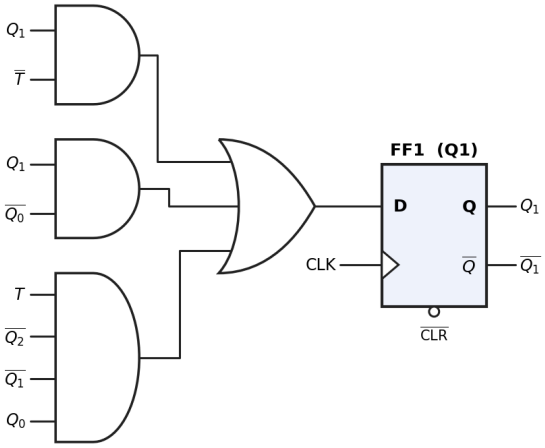
The circuit is drawn in two parts. Figure 3 shows the three D flip-flops with their next-state logic; Figure 4 shows the output decoder that drives the lamps. Signal names are used instead of long wires (each label connects to the flip-flop output of the same name).

Next-state logic and state register (Moore FSM)

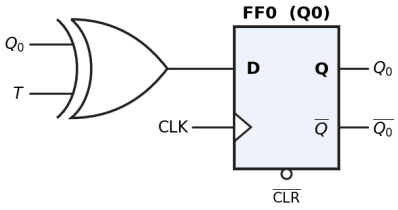
D2 = Q2·T' + Q2·Q0' + T·Q1·Q0



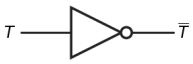
D1 = Q1·T' + Q1·Q0' + T·Q2'·Q1'·Q0



D0 = Q0 ⊕ T (toggle when T = 1)



One NOT gate makes T' for the logic above:



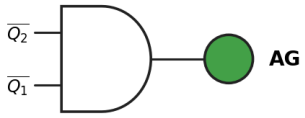
Notes: a bar over a signal means NOT. Complemented flip-flop outputs (Q') come from the Q-bar pin of each flip-flop. All flip-flops share CLK. Active-low CLR (asynchronous) of all three is tied to RESET (power-on reset to state 000).

Figure 3 – State register and next-state logic of the Moore FSM.

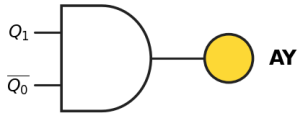
Output decoder (Moore: inputs are Q2, Q1, Q0 only)

ROAD A (main road)

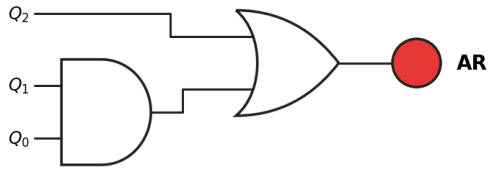
A_G = Q2'·Q1'



A_Y = Q1·Q0'

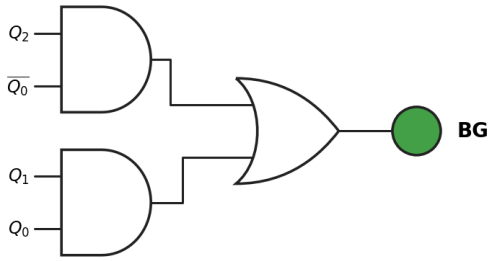


A_R = Q2 + Q1·Q0

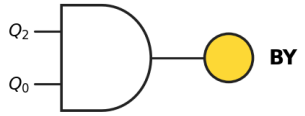


ROAD B (side road)

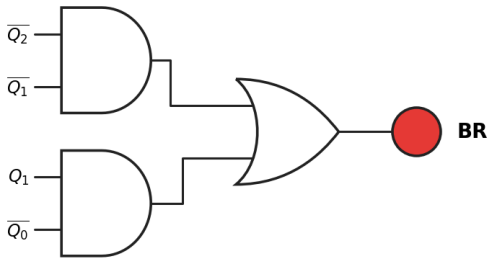
B_G = Q2·Q0' + Q1·Q0



B_Y = Q2·Q0



B_R = Q2'·Q1' + Q1·Q0'



Each lamp is an LED with a series resistor to ground (active-high output).

Figure 4 – Moore output decoder (depends on $Q_2Q_1Q_0$ only).

1.8 Unused states and reset behaviour

State	Next if T = 0	Next if T = 1	Lamps ($A_RA_YA_G$ $B_RB_YB_G$)
110	110	111	110 101 (invalid pattern)
111	111	100	100 011 (invalid pattern)

Table 4 – What the minimised logic does in the two unused states.

Using don't-cares gives smaller logic, but it means states 110 and 111 do not show a valid lamp pattern. They cannot be reached in normal operation because the asynchronous **RESET** clears all three flip-flops to 000 at power-on. If noise ever forced the machine there anyway, the equations still recover it: 110 → 111 → 100 (S4) with two T = 1 pulses, and 111 → 100 with one (see the check below). A real installation would also blank the lamps in those two states; that needs a few more gates and was left out to keep the minimised logic asked for.

Part 2 – MOD-6 synchronous counter

2.1 Choice of N and requirements

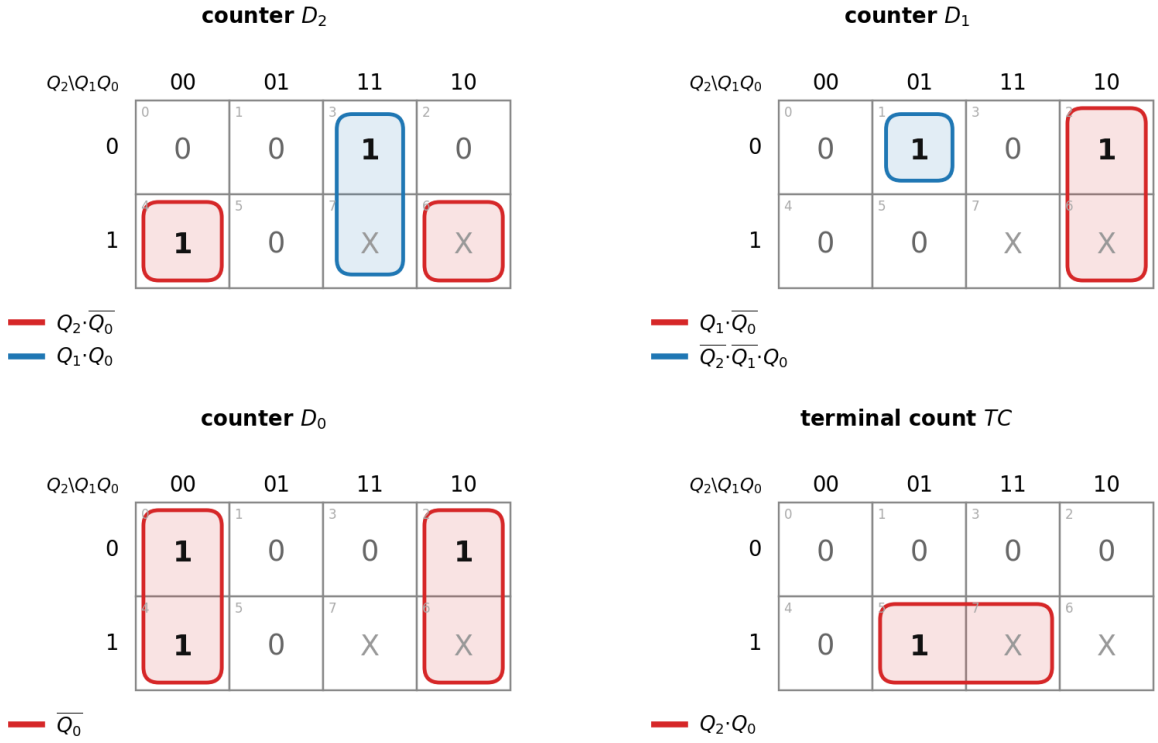
N = 6 was chosen (allowed range 5–10). A MOD-N counter needs $\lceil \log_2 N \rceil$ flip-flops, so 3 are needed ($Q_2Q_1Q_0$). It counts 0, 1, 2, 3, 4, 5 and then returns to 0; the codes 110 (6) and 111 (7) are unused and become don't-cares. It is a **synchronous** counter: all three flip-flops share the same clock and change together, with the next-state logic in front of the D inputs. The terminal-count output TC is 1 when the counter is at its last state (5) and tells the traffic-light FSM that six clocks have elapsed.

2.2 State table

Count	$Q_2Q_1Q_0$	Next count	$D_2D_1D_0$	TC
0	000	1	001	0
1	001	2	010	0
2	010	3	011	0
3	011	4	100	0
4	100	5	101	0
5	101	0	000	1
6 (unused)	110	X	X X X	X
7 (unused)	111	X	X X X	X

Table 5 – MOD-6 counter state table.

2.3 K-maps and equations



$$D_2 = Q_2\cdot Q_0' + Q_1\cdot Q_0 \qquad D_1 = Q_1\cdot Q_0' + Q_2'\cdot Q_1'\cdot Q_0 \qquad D_0 = Q_0' \qquad TC = Q_2\cdot Q_0$$

2.4 Circuit

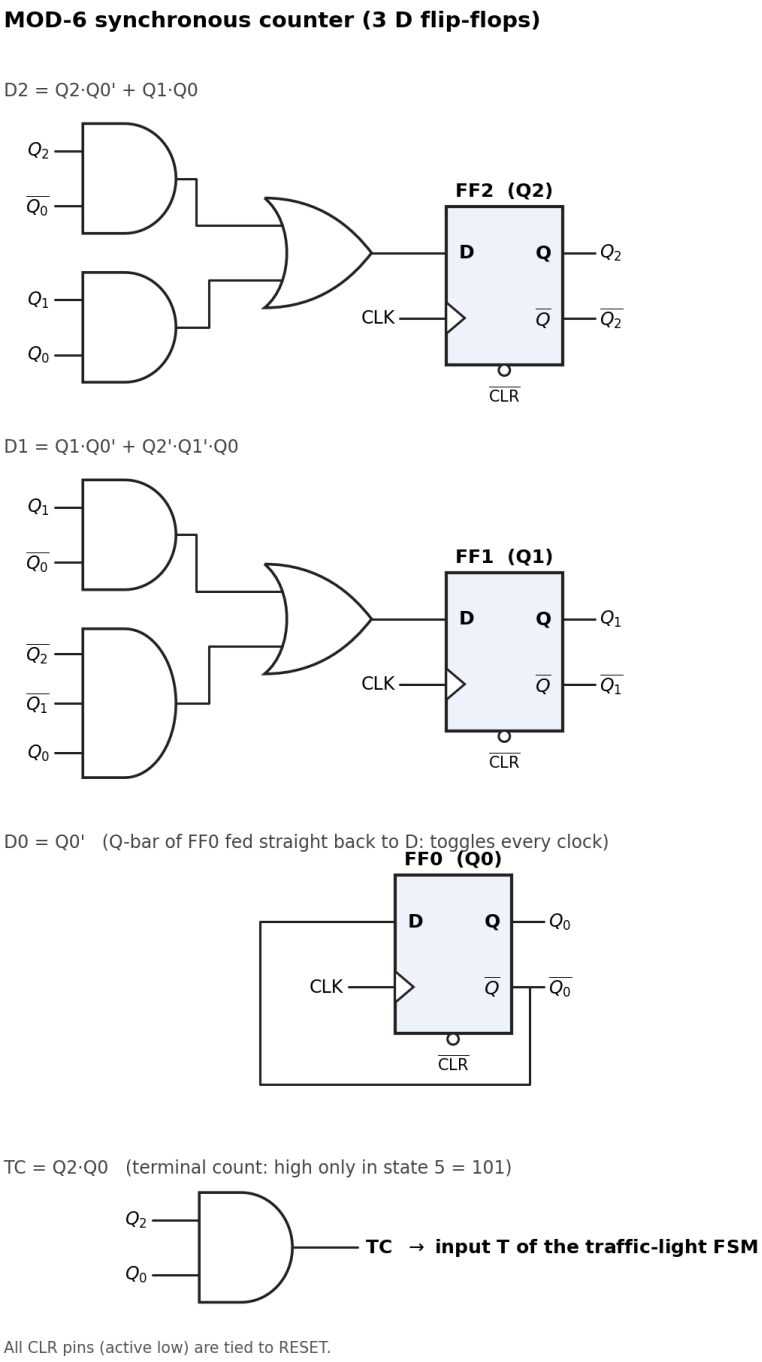


Figure 5 – MOD-6 synchronous counter with terminal-count output.

2.5 Terminal-count logic

TC must be 1 only in the last valid state, 5 = 101. Among the valid states 0 to 5, only state 5 has both $Q_2 = 1$ and $Q_0 = 1$ (state 4 = 100 has $Q_0 = 0$, and states 0–3 have $Q_2 = 0$), so a single AND gate, **$TC = Q_2 \cdot Q_0$**, is enough. State 7 = 111 would also give TC = 1, but it is an unused state that never occurs in normal counting.

2.6 Wrap-around proof

(a) The last state returns to zero. In state 5 ($Q_2Q_1Q_0 = 1\ 0\ 1$) the equations give:

$$D_2 = Q_2Q_0' + Q_1Q_0 = 1\cdot0 + 0\cdot1 = 0 \quad D_1 = Q_1Q_0' + Q_2'Q_1'Q_0 = 0\cdot0 + 0\cdot1\cdot1 = 0 \quad D_0 = Q_0' = 0$$

so the next state is 000, i.e. count 5 → count 0. This is the wrap-around.

(b) The sequence is a single closed loop of length 6. Evaluating the equations for each state gives 0 → 1 → 2 → 3 → 4 → 5 → 0. Each state has exactly one successor, the loop closes at state 0, and no state is skipped or repeated inside it, so the counter has period exactly N = 6 and TC = 1 occurs once per 6 clocks (in state 5).

(c) No lock-up (self-starting). From the unused states: 110 → 111 → 100 and 111 → 100. State 100 = 4 is inside the loop, so any start-up state enters the normal sequence within two clocks. (RESET to 000 is used anyway so the two counters always start together.)

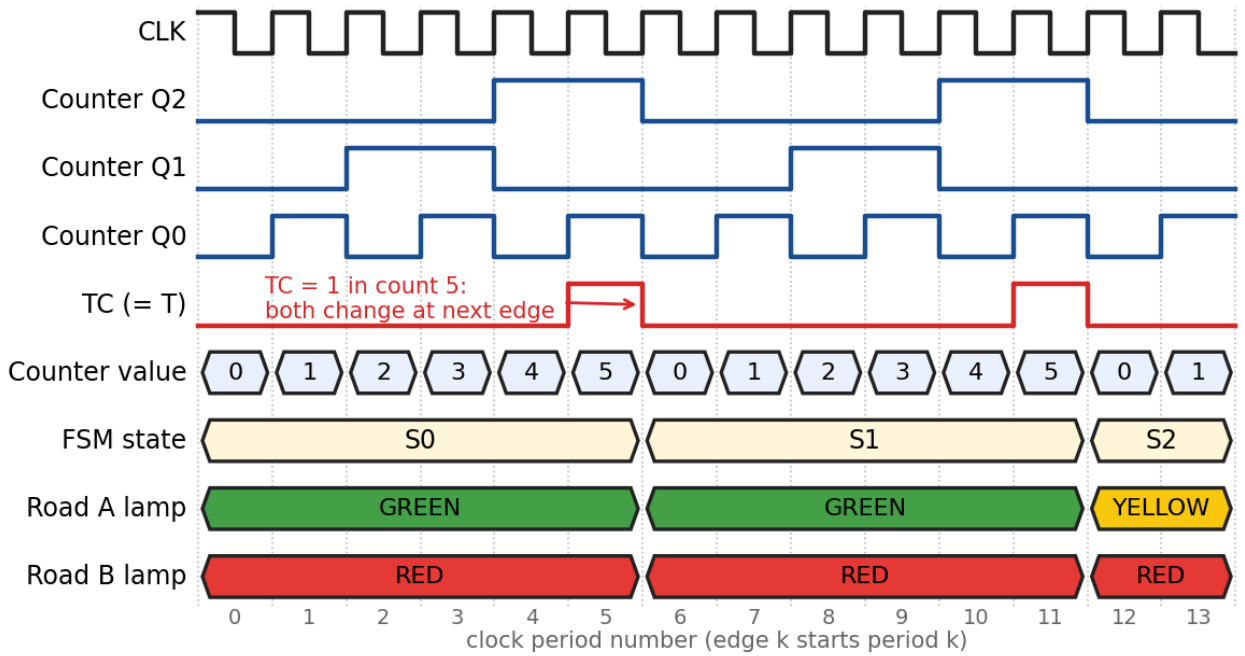


Figure 6 – Counter waveforms: 0…5, wrap to 0 and TC = 1 in count 5; the FSM state changes on the same edge as the wrap.

Part 3 – The two circuits working together

TC of the counter is connected to input T of the FSM and both circuits use the same clock and RESET. During count 5 TC = 1, so at the next clock edge the counter wraps to 0 **and** the FSM moves to its next state. Every state therefore lasts exactly six clock periods (Figure 6). The full 36-clock cycle is shown in Figure 7.

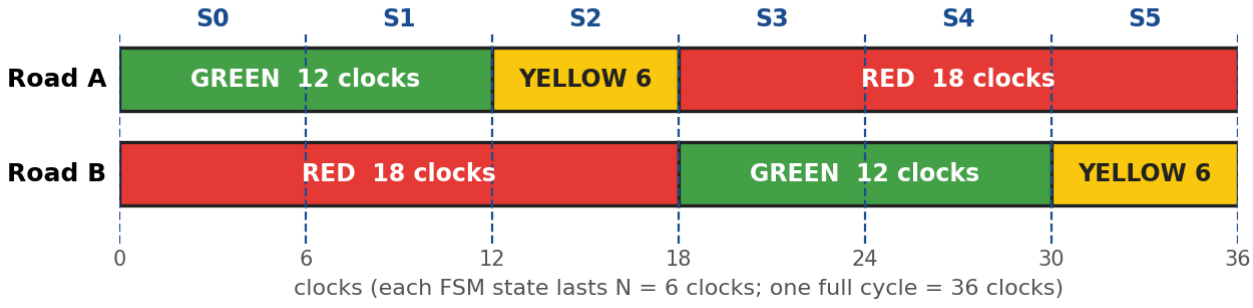


Figure 7 – One complete cycle of the controller (36 clocks).

Phase	States	Clocks	At 1 Hz clock	At 2 Hz clock
Road A green (B red)	S0, S1	12	12 s	6 s
Road A yellow (B red)	S2	6	6 s	3 s
Road B green (A red)	S3, S4	12	12 s	6 s

Road B yellow (A red)	S5	6	6 s	3 s
Total cycle		**36**	**36 s**	**18 s**

Table 6 – Phase durations. Changing N changes every phase length by the same factor.

Block	Flip-flops	Gates (2-input unless stated)
MOD-6 counter	3 D	4 AND, 1 AND-3, 2 OR
FSM next-state logic	3 D	4 AND, 1 AND-3, 1 AND-4, 2 OR-3, 1 XOR, 1 NOT
FSM output decoder	–	8 AND, 3 OR

Table 7 – Component count for a hardware build (6 D flip-flops, e.g. three 74HC74 chips, plus the gates above).

Part 4 – Reflection: why Moore instead of Mealy?

I chose a Moore machine because the lamps should depend on the state only, never directly on the input T. In a Mealy machine the outputs are functions of both the state and T. Here T comes from combinational logic (the AND gate that decodes the counter's terminal count), so a short glitch on T would pass straight through the output logic to the lamps and could change a lamp in the middle of a clock period. In the Moore design the six lamp signals are decoded only from the three state flip-flops, so the lamps can change only just after a clock edge, whatever T does. This matters for a traffic light, where the one thing that must never happen is both roads showing green. Moore also makes checking easy: every lamp pattern belongs to exactly one state, so the safety rule can be verified by reading six rows of Table 2. The price is that a state is needed for each lamp pattern and time slot (a Mealy machine might need fewer states), and that outputs react one clock later than the input. For a controller whose phases last seconds, one more state and a one-clock delay cost nothing, so Moore is the better trade. (The decoder gates can still show a few nanoseconds of hazard when two state bits change at once, for example 011 → 100; if that mattered the six outputs could be passed through a register.)

Appendix – Verification

All equations in this document were checked exhaustively with a short Python program (`verify_design.py`) that applies the exact gate equations to every state and compares the result with the state tables. The K-map groups were also checked automatically (no group covers a 0, every 1 is covered). Output of the program:

```
PART A - MOD-6 counter
  sequence from 000: [0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 0]
  TC = 1 only in valid state(s): [5]
  wrap-around: next(101) = 000 (must be 000)
  unused state 110 -> path ['110', '111', '100']
  unused state 111 -> path ['111', '100']

PART B - Moore FSM
  S0 A-green / B-red     out(AR,AY,AG,BR,BY,BG) = (0, 0, 1, 1, 0, 0)
  S1 A-green / B-red     out(AR,AY,AG,BR,BY,BG) = (0, 0, 1, 1, 0, 0)
  S2 A-yellow / B-red    out(AR,AY,AG,BR,BY,BG) = (0, 1, 0, 1, 0, 0)
  S3 A-red / B-green     out(AR,AY,AG,BR,BY,BG) = (1, 0, 0, 0, 0, 1)
  S4 A-red / B-green     out(AR,AY,AG,BR,BY,BG) = (1, 0, 0, 0, 0, 1)
  S5 A-red / B-yellow    out(AR,AY,AG,BR,BY,BG) = (1, 0, 0, 0, 1, 0)
  unused 110 with T=1 -> ['110', '111', '100']
  unused 111 with T=1 -> ['111', '100']

PART C - counter + FSM together (one clock = 1 step)
  (state, clocks spent): [(0, 6), (1, 6), (2, 6), (3, 6), (4, 6), (5, 6)]
  full cycle = 6 states x 6 clocks = 36 clocks (A green 12, A yellow 6, B green 12, B yellow 6)

RESULT: ALL CHECKS PASSED
```